

LoViF 2026 Real-World All-in-One Image Restoration Challenge Factsheet

Clean DiffUIR with MoCE-Prompt Residual Refinement

Ruibo Zhang

University of Science and Technology of China
ruibo Zhang@mail.ustc.edu.cn

Xinyan Liu

Harbin Institute of Technology
xinyanliu@hit.edu.cn

Dexiang Hong

University of Science and Technology of China
hongdexiang@mail.ustc.edu.cn

Shengeng Tang

Hefei University of Technology
tangsg@hfut.edu.cn

Weidong Chen

University of Science and Technology of China
chenweidong@ustc.edu.cn

1. Team Details

- **Challenge full name:** LoViF 2026 - Real-World All-in-One Image Restoration.
- **Team name:** ustc_pilab.
- **Team ID:** Not used / not applicable.
- **Team leader:** Ruibo Zhang, ruibo Zhang@mail.ustc.edu.cn.
- **Team members:** Dexiang Hong (hongdexiang@mail.ustc.edu.cn), Xinyan Liu (xinyanliu@hit.edu.cn), Shengeng Tang (tangsg@hfut.edu.cn), and Weidong Chen (chenweidong@ustc.edu.cn).
- **Affiliations:** University of Science and Technology of China; Harbin Institute of Technology; Hefei University of Technology.
- **Sponsor affiliation:** None.
- **Team website / code repository:** https://github.com/Zrb3786/all_in_one_restore.
- **Codabench team name:** ustc_pilab.
- **Best final entry:** Clean-310 + MoCE-Prompt Refiner v2-150k, final score 28.47.
- **Source code and executable link:** https://github.com/Zrb3786/all_in_one_restore.
- **Single model checkpoint link:** https://drive.google.com/file/d/1VlfAuA-jeZQEY-_moxCXAkhrEDHI-Wsj/view?usp=drive_link.
- **Submitted result link:** placeholder release link: https://github.com/Zrb3786/all_in_one_restore/releases/tag/lovif-final-20260707.

- **LaTeX factsheet source link:** placeholder Overleaf link: <https://www.overleaf.com/read/gxzbgnhjjpgdg#c6f57e>.

2. Method Overview

We propose a unified all-in-one restoration system for the LoViF 2026 real-world all-in-one image restoration track. The model is designed around two empirical observations from our development process. First, the publicly released DiffUIR universal checkpoint has strong motion-blur restoration behavior, but its raw output is conservative on several LoViF low-light and real-weather cases. Second, direct task-weighted fine-tuning can improve low-light, rain, haze, and snow, but over-aggressive losses tend to damage text, edges, and blur details. To balance these effects, our final system keeps a fine-tuned DiffUIR model as a stable universal backbone and adds a small residual refiner that learns only conservative, local corrections.

The final model is a two-stage restoration network internally packed into a single checkpoint. It first restores the low-quality input image with a clean fine-tuned DiffUIR backbone, and then applies a lightweight MoCE-Prompt residual refiner to correct remaining low-light and weather degradation artifacts while preserving the stability of the DiffUIR output. At inference time, all input images are processed by exactly the same pipeline without degradation labels, file-index heuristics, task-specific checkpoints, or explicit external routing.

The final inference flow is:

$$\hat{y}_0 = F_{\text{DiffUIR}}(x), \quad \hat{y} = R_{\text{MoCE}}(x, \hat{y}_0, \hat{y}_0 - x), \quad (1)$$

Table 1. Administrative information to be finalized before submission. The table is intentionally kept in the factsheet so that organizers can quickly locate the Codabench identity, source-code package, result package, and final scoring information.

Item	Content
Team name	ustc_pi_lab
Codabench account / team ID	ustc_pi_lab; final submission ID: pending / not listed
Team leader	Ruibo Zhang, ruibo Zhang@mail.ustc.edu.cn
Affiliations	USTC, HIT, and HFUT
Source package	https://github.com/Zrb3786/all_in_one_restore
Single model checkpoint	https://drive.google.com/file/d/1VlfAuA-jeZQEY-_moxCXAkhrEDHI-Wsj/view?usp=drive_link
Full test results	https://github.com/Zrb3786/all_in_one_restore/releases/tag/lovif-final-20260707
LaTeX source package	https://www.overleaf.com/read/gxzbgnhjpgdg#c6f57e
Final score	Final Score 28.47; PSNR(Y) 22.65; SSIM(Y) 0.77; LPIPS 0.38

where x is the degraded input image, F_{DiffUIR} is the clean-310 DiffUIR backbone, R_{MoCE} is the proposed residual refiner, $\hat{y}_0$ is the intermediate restoration, and $\hat{y}$ is the final restoration result.

Submission compliance. The submitted parameter file is a single checkpoint, which contains both the DiffUIR clean-310 backbone parameters and our MoCE-Prompt residual refiner parameters. The executable inference script loads this single file and performs the two sequential restoration stages internally. The output images are saved in JPEG format with quality 96, no chroma subsampling, and the same spatial size and base file name as the input.

3. Network Architecture

3.1. Stage 1: Clean DiffUIR Backbone

The first stage uses DiffUIR [1] as the universal image restoration backbone. We initialize from the publicly available DiffUIR universal checkpoint `model-300.pt`. We then fine-tune it on the LoViF training split using the official DiffUIR training objective, without our previous auxiliary loss modifications. In our final backbone, the checkpoint after fine-tuning to milestone 310 is used. We refer to this model as **clean-310**.

The clean-310 backbone is applied uniformly to all five degradation types in LoViF: motion blur, low-light, haze, rain, and snow. It gives a stable all-in-one restoration baseline and improves low-light and weather cases over the original universal checkpoint, while retaining a reasonable level of blur restoration.

3.2. Stage 2: MoCE-Prompt Residual Refiner

The second stage is our custom lightweight residual refiner. It is designed as a small post-restoration network that corrects only the residual errors left by the clean-310 backbone. The refiner receives a 9-channel tensor composed of

the original degraded image, the clean-310 restoration, and the residual between them:

$$z = \text{Concat}(x, \hat{y}_0, \hat{y}_0 - x). \quad (2)$$

This representation is important in practice: the clean-310 output provides a strong restored anchor, while the residual term highlights where the backbone still differs from the input and therefore where additional correction may be beneficial.

Architecturally, the refiner follows the implementation used in our codebase. It contains four main parts:

1. **Feature stem.** A shallow convolutional embedding layer maps the 9-channel input into a compact feature space (width 32 in the final submission model).
2. **MoCE-Prompt trunk.** A stack of 4 restoration blocks processes the embedded features. Each block has a shared feature-update pathway together with several lightweight experts of different effective complexities/receptive fields. This part is inspired by all-in-one restoration MoE-style designs, but is kept much smaller than the main DiffUIR backbone.
3. **Prompt/gating branch.** Global pooled features are sent to a small prompt/gating MLP that predicts sample-adaptive expert weights. This allows the same refiner to softly emphasize different experts for blur, low-light, haze, rain, or snow cases, without using explicit degradation labels at inference time.
4. **Residual and mask heads.** The last layer predicts (i) a residual correction map r and (ii) a correction mask m . The mask limits where the residual is applied, which makes the module conservative and helps preserve the structure already recovered by clean-310.

The final result is obtained by a conservative residual correction:

$$\hat{y} = \text{clip}(\hat{y}_0 + \alpha \cdot m \odot r, 0, 1), \quad (3)$$

Table 2. Summary of the final all-in-one method. The same model and inference pipeline are used for all LoViF degradation types.

Component	Implementation	Role in the final system
Backbone	DiffUIR clean-310	Universal restoration stage initialized from the released DiffUIR checkpoint and fine-tuned on LoViF using official DiffUIR loss. Provides the main image restoration and strong baseline behavior.
Residual refiner	MoCE-Prompt Refiner v2-150k	Lightweight post-backbone correction module. It takes the input image, clean-310 output, and their residual, and predicts a small masked residual update.
Single checkpoint	Single packed .pt file	Contains both the clean-310 DiffUIR state and the refiner state. No external degradation classifier or per-task checkpoint selection is used. The exact file name is given in Tab. 1.
Output format	JPEG quality 96	The final inference script saves RGB JPEG images with quality 96, no chroma subsampling, and the same file names and sizes as the inputs.

where r is the predicted residual, m is the learned correction mask, and α is the residual scale. In the submitted model, $\alpha = 0.15$. This design starts from a strong and stable clean-310 restoration and learns only the necessary local corrections, reducing the risk of damaging structures, text edges, and motion-blur restoration. The exact validation loader

Why this refiner is lightweight but useful. Compared with directly making the DiffUIR backbone more aggressive, the refiner is easier to train, cheaper to continue from cached backbone outputs, and more controllable. Empirically, this design improved low-light and weather cases while remaining much safer than directly re-optimizing the entire diffusion backbone with strong weather-oriented losses.

All-in-one behavior. Although the refiner contains internal prompt/expert modules, it does not use explicit degradation labels during inference. The same network, same checkpoint, and same two-stage pipeline are used for all input images.

4. Training Data and Pre-processing

4.1. Training data

We use the official LoViF all-in-one restoration training set, which provides paired low-quality and ground-truth images for the five degradation categories: blur, low-light, haze, rain, and snow. We created a deterministic seed-42 split for model development:

- **Training split:** approximately 90% of each degradation category;
- **Validation split:** approximately 10% of each degradation category.

The public/final test images are not used for training. They are processed only for final inference.

4.2. External data and pretrained models

We did not add any additional image pairs from external datasets during our LoViF fine-tuning or refiner training.

However, our DiffUIR backbone is initialized from the publicly released DiffUIR universal checkpoint, which contains prior restoration knowledge learned by the original DiffUIR authors. The final clean-310 backbone is obtained by fine-tuning this checkpoint on the LoViF training data split.

5. Training Strategy

5.1. Clean DiffUIR fine-tuning

The clean-310 backbone is trained from the DiffUIR milestone-300 checkpoint using the official DiffUIR loss. We use the LoViF seed-42 training split with a paired dataset loader that ensures each low-quality image is paired with the ground-truth image of the same file name. We do not use task-dependent inference or external degradation classification.

- **Initialization:** public DiffUIR universal model-300.pt.
- **Training target:** milestone 310.
- **Patch size:** 256×256 .
- **Batch size:** 8.
- **Gradient accumulation:** 2.
- **Learning rate:** 1×10^{-5} .
- **Sampling timesteps:** 3 for the DDIM sampling configuration used in inference.
- **Low-light histogram equalization:** disabled for the final clean training, to keep training and inference consistent on raw inputs.

5.2. Clean-310 output cache

To train the refiner efficiently, we first run clean-310 on all training and validation images and cache the intermediate restorations. During refiner training, each sample contains:

$$(x, y, \hat{y}_0), \quad (4)$$

where x is the input image, y is the ground truth, and $\hat{y}_0$ is the clean-310 output.

5.3. MoCE-Prompt refiner training

The refiner is trained in two phases. First, a conservative base refiner is trained for 100k iterations. Then, a weather-

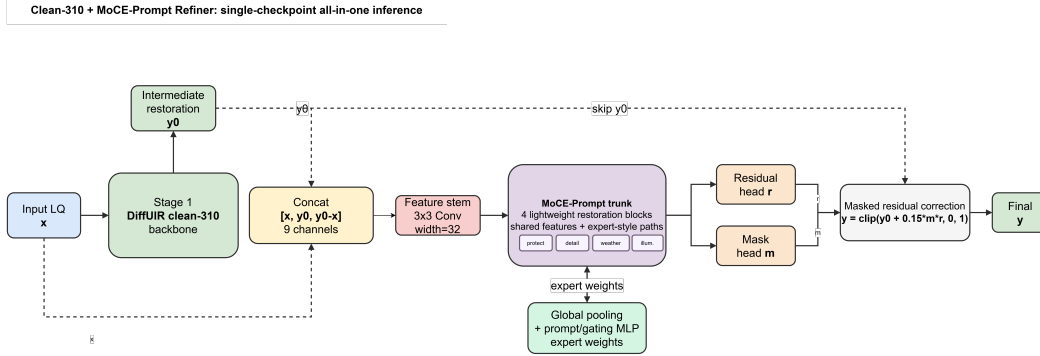


Figure 1. Architecture sketch of the final submitted model, drawn to match the structure used in our code. A low-quality input image is first restored by the clean-310 DiffUIR backbone. The refiner then takes the concatenation of the input image, the clean-310 output, and their residual (9 channels total). A shallow stem feeds a lightweight MoCE-Prompt trunk with sample-adaptive expert weighting. Finally, residual and mask heads predict a conservative correction that is added to the clean-310 output. The two stages are packed into one single checkpoint for submission.

enhanced continuation stage is trained to 150k iterations using a modified loss that increases the training emphasis on haze and snow residual regions while preserving blur and structural stability.

- **Input:** concatenation of input image, clean-310 restoration, and their residual.
- **Patch size:** 256×256 .
- **Architecture width:** 32.
- **Number of blocks:** 4.
- **Batch size:** 8.
- **Optimizer:** Adam/AdamW-style optimizer as implemented in the provided training script.
- **Base refiner training:** 100k steps, learning rate 1×10^{-4} .
- **Weather continuation:** 100k to 150k steps, learning rate 5×10^{-5} .
- **Residual scale:** 0.15.
- **Final refiner checkpoint:** `refiner_step150000.pt`.

6. Loss Function

The refiner loss combines reconstruction, structure preservation, base-output consistency, and task-aware weighting. During training we know which subfolder a sample comes from, so we use the degradation category only to weight losses. At inference time, no category label is used.

The overall training objective can be summarized as

$$\mathcal{L} = \sum_k w_k \mathcal{L}_k, \quad (5)$$

where the active weights w_k depend on the degradation category during training only. The main terms are listed below.

The loss includes:

- Charbonnier reconstruction loss;
- gradient and Laplacian losses for structural preservation;
- luminance and exposure losses for low-light samples;
- change-weighted reconstruction loss, where the soft weight is derived from $|x - y|$, to emphasize degraded regions;
- snow-cue weighted loss based on high-brightness/low-saturation and local contrast cues for snow samples;
- base-keep and residual-magnitude regularization to prevent the refiner from over-editing stable clean-310 outputs.

For blur samples, the base-keep weight is higher to avoid degrading the clean-310/base restoration. For haze and snow samples, the change-weighted and snow-cue losses are strengthened, because we observed that the model tends to leave partial haze/snow as if they were part of the scene. For rain samples, the loss is more conservative because the clean-310 and intermediate refiner already remove many rain artifacts effectively.

In implementation terms, the weather-v2 continuation mainly increases the supervision on regions where the

Table 3. Training configuration used by the final model. The values correspond to the final selected clean-310 + refiner v2-150k submission candidate.

Stage	Main settings	Notes
Clean DiffUIR fine-tuning	init: public DiffUIR model-300.pt; target: model-310.pt; patch: 256×256 ; batch: 8; grad accum.: 2; lr: 1×10^{-5}	Uses official DiffUIR loss and LoViF paired seed-42 train split. Low-light histogram equalization is disabled to keep train/test inputs consistent.
Clean-310 cache	Run clean-310 on seed-42 train/val images	Cached outputs are used as fixed backbone outputs for efficient refiner training.
Base refiner training	width: 32; blocks: 4; residual scale: 0.15; patch: 256×256 ; batch: 8; lr: 1×10^{-4} ; steps: 100k	Learns conservative residual corrections over clean-310.
Weather-v2 continuation	resume from 100k; lr: 5×10^{-5} ; final step: 150k	Strengthens haze/snow change-region losses while preserving base-output consistency.
Final checkpoint	clean-310 + refiner step 150k	Packed into one parameter file for submission.

input-ground-truth discrepancy is large, and on pseudo-snow cue regions characterized by high brightness, low saturation, and local contrast. This is the main reason why the 150k refiner is more proactive on haze/snow than the earlier 100k model, while still preserving the clean-310 structure on blur and text-heavy regions.

7. Testing and Submission

For final testing, the single checkpoint is loaded by `infer_single_ckpt_jpeg96.py`. For each test image, the script performs:

1. DiffUIR clean-310 restoration;
2. MoCE-Prompt residual refinement;
3. JPEG saving with `quality=96`, `subsampling=0`, and `optimize=True`.

The output file name is kept identical to the input file name, and the output size is kept identical to the input size. The output images are RGB JPEG files.

8. How to Reproduce the Comparison Tables

Because the challenge evaluation is based on the organizer server, we separate *internal validation* from *final Codabench submission*. Table 4 is intended for the internal seed-42 validation split, while Table 5 is intended for the final organizer-reported numbers.

Recommended workflow for Table 4. We recommend generating three result folders on the seed-42 validation split: (i) DiffUIR base-300, (ii) DiffUIR clean-310, and (iii) clean-310 + refiner v2-150k. The intermediate refiner-100k comparison is intentionally omitted from the final factsheet table. In our released source package, this can be reproduced with commands of the following form:

```
# 1) DiffUIR base-300 on the seed42 val split
python infer_lovif_flat.py \
```

```
--input_dir /path/to/
CVPR26_LoViF_AIO_split_seed42/val/LQ_all \
--output_dir /path/to/results/base300 \
--ckpt_dir /path/to/ckpt_universal/diffuir \
--milestone 300 --sampling_timesteps 3

# 2) DiffUIR clean-310 on the same validation
split
python infer_lovif_flat.py \
--input_dir /path/to/
CVPR26_LoViF_AIO_split_seed42/val/LQ_all \
--output_dir /path/to/results/clean310 \
--ckpt_dir /path/to/diffuir_clean310_ckpt_dir \
--milestone 310 --sampling_timesteps 3

# 3) Final refiner v2-150k
python infer_refiner_clean310_moce.py \
--input_dir /path/to/
CVPR26_LoViF_AIO_split_seed42/val/LQ_all \
--base_dir /path/to/results/clean310 \
--output_dir /path/to/results/refiner150k \
--refiner_ckpt /path/to/refiner_step150000.pt
--use_ema

# 4) Evaluate by category and overall
python tools_clean310_moce/eval_refiner_val.py \
--gt_root /path/to/
CVPR26_LoViF_AIO_split_seed42/val \
--pred_root /path/to/results/refiner150k
```

The exact validation loader layout can differ depending on how the user stores the seed-42 split. If the released helper shell scripts are preferred, `tools_clean310_moce/infer_val_tasks_clean310_refiner.sh` can be adapted to the local paths and then followed by `eval_refiner_val.py`. The resulting PSNR/SSIM/LPIPS values can be copied into Table 4.

Recommended workflow for Table 5. For the final test set, we use the single packed checkpoint and the unified inference script:

```
python infer_single_ckpt_jpeg96.py \
```

Table 4. Internal seed-42 validation metrics. Each cell reports PSNR(Y) / SSIM(Y) / LPIPS. The refiner-100k row is intentionally omitted.

Model	Blur	Low-light	Haze	Rain	Snow
DiffUIR base-300	17.883/0.368/0.564	8.683/0.198/0.675	15.606/0.619/0.416	25.654/0.767/0.351	23.155/0.785/0.182
DiffUIR clean-310	18.228/0.379/0.622	12.383/0.498/0.504	17.546/0.675/0.378	33.762/0.939/0.200	26.018/0.812/0.168
Clean-310 + MoCE-Prompt Refiner	18.230/0.379/0.622	16.676/0.688/0.448	20.149/0.695/0.358	33.740/0.938/0.202	26.146/0.813/0.167

Table 5. Final Codabench/test result of the selected single-checkpoint submission.

Submission	PSNR(Y)	SSIM(Y)	LPIPS	Final score	Notes
Clean-310 + MoCE-Prompt Refiner v2-150k	22.65	0.77	0.38	28.47	Final single-checkpoint submission.

```
--single_ckpt checkpoints/
  lovif_clean310_refiner_v2_150k_single.pt \
--input_dir /path/to/final_test \
--output_dir /path/to/final_results_jpeg96 \
--code_root . --sampling_timesteps 3 --
  image_size 256
```

The saved images are then zipped and submitted to Codabench. The organizer-reported PSNR(Y), SSIM(Y), LPIPS, and final score can be filled into Table 5. The final Codabench result has been filled in Table 5.

9. Experimental Observations

On the seed-42 internal validation and smoke-test images, the clean-310 backbone improves low-light, rain, haze, and snow examples compared with the original DiffUIR universal checkpoint. Motion blur remains challenging; the original DiffUIR checkpoint is often stronger on blur, so the refiner is trained conservatively to avoid damaging the clean-310 blur results.

The MoCE-Prompt refiner further improves selected low-light/weather cases and allows stronger correction of haze and snow residuals. We observed that overly aggressive weather losses can blur text and structure, so the final submission uses a conservative residual scale and base-keep regularization. From our smoke-test comparisons, the most consistent gains came from low-light enhancement and partial haze/snow clean-up, while the most difficult trade-off remained preserving motion-blur quality without sacrificing weather restoration. This observation directly motivated our final choice of a stable clean-310 backbone plus a lightweight residual-only second stage, instead of a more aggressive end-to-end all-in-one fine-tuning strategy.

10. Ensembles and Fusion Strategies

We do not use test-time ensembling, multiple submitted checkpoints, or degradation-specific external routing. The final submitted system is a single unified model checkpoint. Internally, the model contains two modules: the clean-310 DiffUIR backbone and our MoCE-Prompt residual refiner.

These modules are always applied sequentially to every input image.

The refiner can be viewed as a learned residual fusion stage over the clean-310 output. This differs from an ensemble because the final output is produced by one fixed forward pipeline and one single checkpoint. No task-specific checkpoint is selected at test time.

11. Reproducibility

The submitted source package contains:

- DiffUIR source code required to instantiate the clean-310 backbone;
- LoViF paired dataset loaders;
- clean DiffUIR fine-tuning script;
- MoCE-Prompt refiner model, loss, training script, and inference script;
- a single-checkpoint inference script;
- the final single checkpoint `lovif_clean310_refiner_v2_150k_single.pt`;
- requirements.txt and README instructions.

The main final inference command is shown below. In the released source package, the checkpoint path points to the packed single model parameter file under `checkpoints/`.

```
python infer_single_ckpt_jpeg96.py
--single_ckpt checkpoints/final.pt
--input_dir /path/to/test
--output_dir /path/to/output
--code_root .
--sampling_timesteps 3
--image_size 256
```

12. References

References

- [1] D. Zheng et al. Selective Hourglass Mapping for Universal Image Restoration Based on Diffusion Model. CVPR, 2024.
- [2] E. Zamfir et al. MoCE-IR: Multiple Complexity Experts for All-in-One Image Restoration. CVPR Workshops, 2024.

Table 6. Qualitative development observations used to select the final model. These observations are based on smoke-test comparison pages and should be replaced or supplemented with final quantitative values when available.

Degradation	Best observed behavior	Main failure mode	Final design response
Motion blur	DiffUIR base/clean checkpoints preserve blur restoration better than aggressive auxiliary fine-tuning.	Strong weather/low-light losses can introduce ghosting or over-smoothing.	Refiner uses high base-keep weight and low residual scale.
Low-light	Clean-310 and refiner improve brightness and color over the public base checkpoint.	Too strong enhancement may shift colors or overexpose.	Luminance/exposure losses are used with conservative residual correction.
Haze	Clean-310 improves many hazy scenes, but some global haze remains.	Strong dehazing can blur text and fine edges.	Weather-v2 continuation increases change/contrast emphasis while keeping edge and base constraints.
Rain	Intermediate fine-tuned models remove rain effectively.	Over-processing may smooth text and brick/line structures.	Rain loss is kept more conservative than haze/snow.
Snow	Some large snow regions are removed, but sparse snow or snow-like texture can remain.	The network may treat snow dots/streaks as scene texture.	Snow-cue and change-weighted losses increase gradients on bright low-saturation local-contrast regions.

- [3] X. Chen et al. LoViF 2026 Challenge on Real-World All-in-One Image Restoration: Methods and Results. CVPR Workshop Report, 2026.
- [4] L. Chen et al. Simple Baselines for Image Restoration. ECCV, 2022.